content was set to `"Wow"`. A `<span>` element with a different style was appended to the `<div>`, and its HTML content was set to `"Even More Wow"`. There's more you can do, but first you need to examine the array object you're binding, and focus on JavaScript arrays and array functions.

ARRAYS AND ARRAY FUNCTIONS

D3 is all about arrays, and so it's important to understand the structure of arrays and the options available to you to prepare those arrays for binding to data. Your array might be an array of string or number literals, such as this:

```
someNumbers = [17, 82, 9, 500, 40];
someColors = ["blue", "red", "chartreuse", "orange"];
```

Or it may be an array of JSON objects, which will become more common as you do more interesting things with D3:

```
somePeople = [{name: "Peter", age: 27}, {name: "Sulayman", age: 24},
{name: "K.C.", age: 49}];
```

One example of a useful array function is `.filter()`, which returns an array whose elements satisfy a test you provide. For instance, here's how to create an array out of `someNumbers` that had values greater than 40:

```
someNumbers.filter(function(el) {return el >= 40});
```

Likewise, here's how you could create an array out of `someColors` with names shorter than five letters:

```
someColors.filter(function(el) {return el.length < 5});
```

The function `.filter()` is a method of an array and accepts a function that iterates through the array with the variable you've named. In this function, you name that variable `el`, and the function runs a test on each value by testing on `el`. When that test evaluates `true`, the element is kept in our new array.

The result of this `.filter()` function, which you can see in figure 1.27, returns either the element or nothing (depending on if it satisfies the test), building a new array consisting only of the elements that do.

```
smallerNumbers = someNumbers.filter(
  function(el) {return el <= 40 ? this : null});

d3.select("body").selectAll("div")
  .data(smallerNumbers)
  .enter()
  .append("div")
  .html(function (d) {return d});
```

The resulting code creates two new divs from your three-value array `smallerNumbers`. (Remember that one div already exists, and so the `.enter()` function doesn't trigger even though data is bound to that existing div.) The contents of the div are the values in your array. This is done through an anonymous function (sometimes referred to in D3 examples as an *accessor*) in your `.html()` function and is another key aspect of D3. Any